Práctica N°2: Inicializacion en ESP32 manejo de ADC y GPIO

Índice

1. Objetivos **2**
- 1.1. Objetivo general . . . 2
- 1.2. Objetivos específicos . . . 2

2. Configuración inicial del entorno y carga de firmware MicroPython **2**
- 2.1. Requisitos previos . . . 3
- 2.2. Conexión del ESP32 al PC . . . 3
- 2.3. Configuración inicial en Thonny . . . 3
- 2.4. Instalación del firmware MicroPython (flasheo) . . . 4
- 2.5. Verificación de instalación . . . 6

3. Explicación del código **6**
- 3.1. Parpadeo simple del LED en GPIO5 . . . 7
- 3.2. Control del LED mediante pulsador (GPIO16) . . . 7
- 3.3. Lectura analógica de un potenciómetro . . . 8

4. Carga y ejecución de programas en el ESP32 con Thonny **9**
- 4.1. Creación del archivo fuente . . . 9
- 4.2. Guardar el archivo en el ESP32 . . . 9
- 4.3. Ejecución del programa . . . 10
- 4.4. Detención del programa . . . 11
- 4.5. Ejecución automática al encender el ESP32 . . . 11
- 4.6. Verificación de salida en la consola . . . 11

5. Resultados **12**
- 5.1. Parpadeo del LED en GPIO5 . . . 12
- 5.2. Control de LED mediante pulsador en GPIO16 . . . 12
- 5.3. Lectura analógica del potenciómetro en GPIO34 . . . 14

1. Objetivos

1.1. Objetivo general

Desarrollar y documentar, paso a paso, una práctica didáctica para programar un ESP32 WROOM-32 con MicroPython usando el entorno Thonny. La práctica cubrirá desde la configuración y carga de firmware hasta la implementación, prueba y verificación de programas básicos de entrada/salida digital, lectura analógica y sensores DHT, fomentando buenas prácticas de cableado, protección (resistencias) y manejo de errores.

1.2. Objetivos específicos

- **Parpadeo LED (blink_pin5.py):** Implementar un programa que haga parpadear un LED conectado a GPIO5 con periodo aproximado de 1 s.
- **Control por pulsador con pull-up interno (led_button_gpio16.py):** Integrar un pulsador conectado a GPIO16 usando la resistencia de pull-up interna del ESP32 y gestionar el rebote por software (debounce). El pulsador deberá encender el LED al presionarse y apagarlo al soltarse.
- **Lectura analógica de potenciómetro (adc_pot_esp32.py):** Configurar un ADC en GPIO34 (ADC1_CH6) con atenuación adecuada (ATTN_11DB) y resolución de 12 bits; leer y escalar a voltaje real (0–3.3 V).
- **Sensor DHT11 (dht11_gpio4.py):** Conectar y leer un sensor DHT11 en GPIO4, implementando manejo de excepciones para lecturas erróneas y periodos de muestreo apropiados.

2. Configuración inicial del entorno y carga de firmware MicroPython

Antes de ejecutar cualquier programa en el ESP32, es necesario instalar el firmware de **MicroPython** dentro del microcontrolador. A continuación se describe el procedimiento completo usando el entorno **Thonny IDE**, disponible para Windows, Linux y macOS. En esta guía se empleará la versión para Windows.

2.1. Requisitos previos

- Computadora con sistema operativo Windows 10 o superior.
- IDE Thonny instalado (https://thonny.org).
- Módulo ESP32 WROOM-32 o tarjeta de desarrollo equivalente.
- Cable USB tipo micro (datos, no sólo carga).
- Firmware oficial de MicroPython para ESP32 (`.bin`), descargable desde: https://micropython.org/download/esp32/.

2.2. Conexión del ESP32 al PC

Conecta el ESP32 a la computadora mediante el cable USB. El sistema operativo debe reconocer el dispositivo como un puerto serial (por ejemplo, `COM3` o similar).

Si el puerto no aparece, puede ser necesario instalar los controladores USB del chip CP2102 o CH340, dependiendo del modelo del módulo ESP32.

2.3. Configuración inicial en Thonny

Abre Thonny y selecciona el intérprete adecuado:

1. Ve al menú `Herramientas → Opciones → Intérprete`.
2. En el campo `Ejecutar en`, selecciona: **MicroPython (ESP32)**.
3. En el campo `Puerto`, elige el puerto COM correspondiente a tu placa.

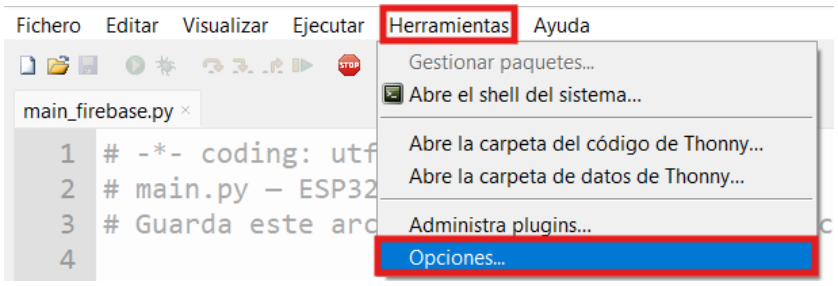


Figura 1: Configuración del intérprete MicroPython (ESP32) en Thonny.

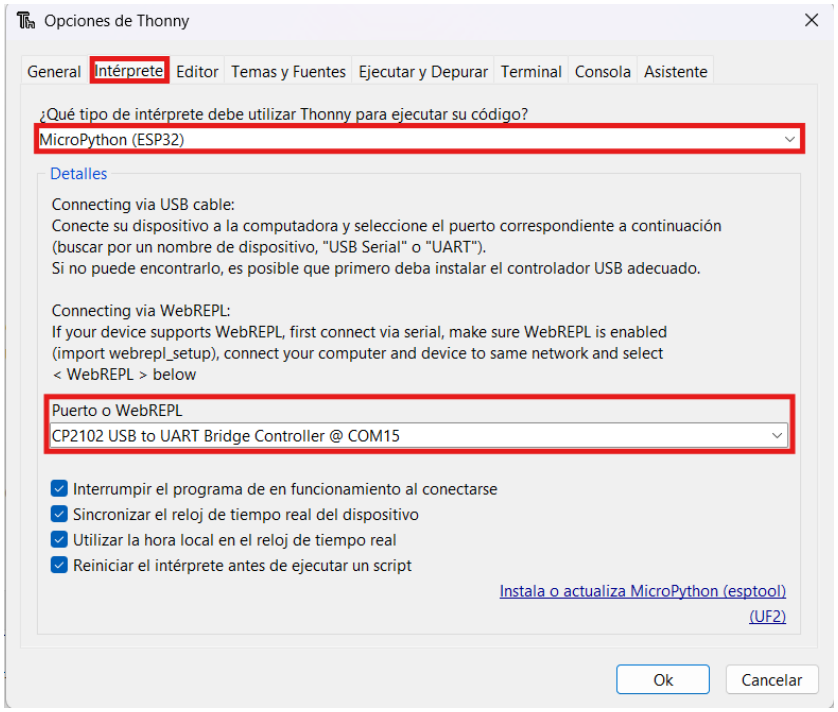


Figura 2: Configuración del intérprete MicroPython (ESP32) en Thonny.

2.4. Instalación del firmware MicroPython (flasheo)

Si el ESP32 no tiene MicroPython instalado o se desea actualizarlo, Thonny permite flashear el firmware directamente:

1. En el mismo cuadro de diálogo de `Opciones → Intérprete`, haz clic en el botón **Instalar o actualizar firmware**.
2. Selecciona el puerto correcto (`COMx`).
3. Pulsa en **Buscar firmware en micropython.org**. Thonny descargará automáticamente la versión estable más reciente.
4. Marca la opción **Borrar completamente el chip antes de instalar**.
5. Finalmente, presiona **Instalar**.

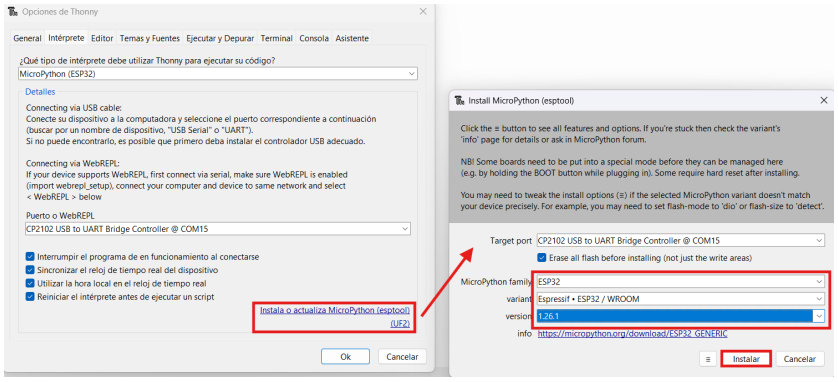


Figura 3: Ventana de instalación o actualización de firmware MicroPython en Thonny.

Durante el proceso, se mostrará el progreso en la barra inferior. Este paso puede tardar entre 30 y 60 segundos. Al finalizar, Thonny mostrará el mensaje `Firmware installation successful`.

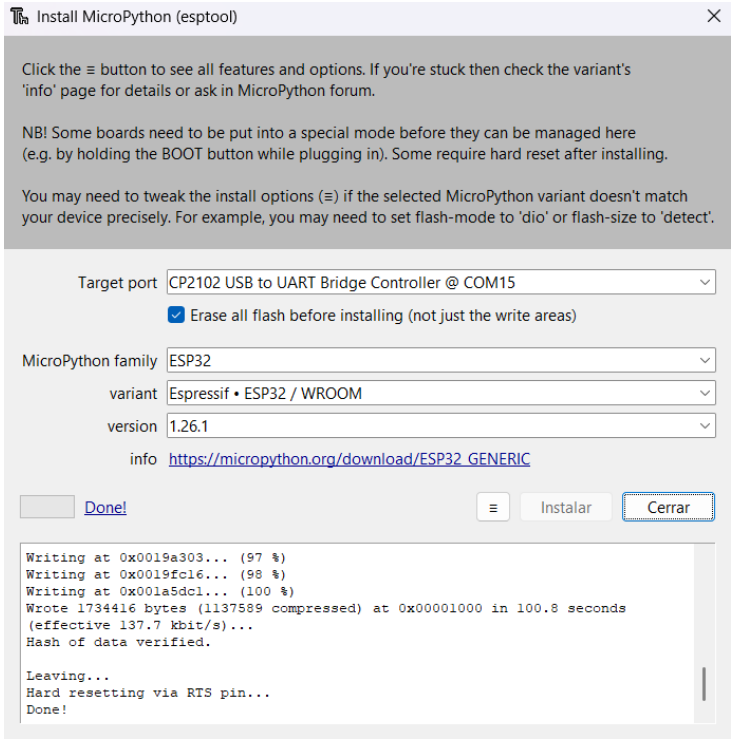


Figura 4: Proceso de flasheo del firmware MicroPython en el ESP32 completado.

2.5. Verificación de instalación

Una vez cargado el firmware:

1. Cierra y vuelve a abrir Thonny.
2. Asegúrate de que el intérprete seleccionado sea nuevamente **MicroPython (ESP32)**.
3. En la consola inferior aparecerá un mensaje similar a:

```
MicroPython v1.22.0 on 2025-01-15; ESP32 module with ESP32
Type "help()" for more information.
>>>
```

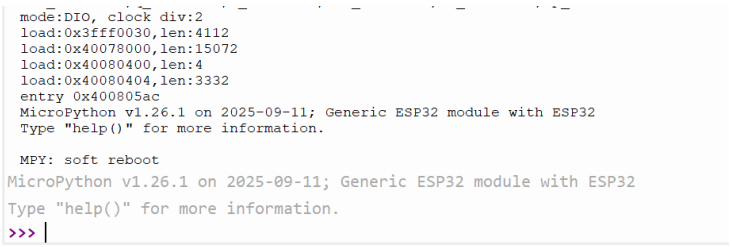


Figura 5: Consola REPL de MicroPython lista para recibir comandos.

3. Explicación del código

En esta sección se analiza el funcionamiento de cada uno de los programas desarrollados en MicroPython para el ESP32. Más que una simple descripción, el propósito es comprender la lógica detrás de cada bloque de código, reconociendo la relación entre el hardware y las instrucciones que lo controlan.

3.1. Parpadeo simple del LED en GPIO5

```
# blink_pin5.py
# MicroPython para ESP32 WROOM-32
# Conecta un LED (con resistencia) al pin GPIO5 (GND compartido)

import machine
import time

print("Prueba ESP32 - parpadeo en GPIO5")

led = machine.Pin(5, machine.Pin.OUT)

while True:
    led.value(not led.value()) # toggle
    time.sleep(1)
```

Listing 1: Parpadeo simple del LED en GPIO5

El código comienza importando las librerías `machine` y `time`, esenciales para acceder al hardware y controlar retardos. La creación del objeto `Pin(5, Pin.OUT)` configura el GPIO5 como salida digital, donde se conecta el LED. Dentro del bucle infinito, el estado del pin se invierte en cada iteración, generando un parpadeo continuo. El retardo de un segundo, introducido por `time.sleep(1)`, hace visible la oscilación. Este ejemplo representa el punto de partida en el control digital del ESP32, donde cada instrucción tiene un reflejo inmediato en el hardware.

3.2. Control del LED mediante pulsador (GPIO16)

```
# led_button_gpio16.py
from machine import Pin
import time

led = Pin(5, Pin.OUT)
button = Pin(16, Pin.IN, Pin.PULL_UP)

DEBOUNCE_MS = 50
last_state = button.value()
last_time = time.ticks_ms()

while True:
    state = button.value()
```

```
    if state != last_state:
        t = time.ticks_ms()
        if time.ticks_diff(t, last_time) > DEBOUNCE_MS:
            last_state = state
            last_time = t
            if state == 0:
                led.on()
            else:
                led.off()
    time.sleep_ms(10)
```

Listing 2: Manejo del LED con un pulsador usando PULL_UP interno

Aquí el programa introduce la interacción entre entrada y salida. El LED conectado al GPIO5 se controla mediante un pulsador en el GPIO16, el cual utiliza una resistencia *pull-up* interna. El código aplica una técnica de *antirrebote* por software, verificando que el tiempo entre cambios de estado supere un umbral definido por `DEBOUNCE_MS`. Así se evita que los rebotes eléctricos del pulsador generen falsas detecciones. El resultado es una respuesta estable: el LED se enciende solo cuando el botón se mantiene presionado. Este ejemplo muestra la transición desde la lógica puramente digital hacia un control más robusto y realista, incorporando consideraciones físicas del hardware.

3.3. Lectura analógica de un potenciómetro

```
# adc_pot_esp32.py
from machine import Pin, ADC
import time

pot = ADC(Pin(34))
pot.atten(ADC.ATTN_11DB)
pot.width(ADC.WIDTH_12BIT)

while True:
    raw = pot.read()
    volts = raw / 4095 * 3.3
    print("ADC raw:", raw, "-> {:.2f} V".format(volts))
    time.sleep(0.5)
```

Listing 3: Lectura analógica desde GPIO34

El uso del módulo `ADC` introduce la lectura analógica en el ESP32. Al conectar un potenciómetro al pin GPIO34, el programa mide valores de 0 a 4095, correspondientes al rango de tensión configurado mediante `ADC.ATTN_11DB`.

El cálculo de voltaje convierte los datos en una escala física más comprensible, mostrando en consola el valor aproximado en voltios. Este ejemplo marca el paso hacia una lectura continua y cuantitativa, donde la información del entorno se transforma en datos numéricos útiles.

4. Carga y ejecución de programas en el ESP32 con Thonny

Una vez instalado el firmware de MicroPython y verificado el funcionamiento básico del intérprete, se pueden crear y ejecutar programas directamente desde el entorno Thonny. Esta sección muestra el procedimiento completo para subir y probar el primer programa: el parpadeo del LED conectado al pin GPIO5.

4.1. Creación del archivo fuente

Abre Thonny y selecciona la pestaña de un nuevo archivo mediante el menú `Archivo → Nuevo`. Escribe el siguiente código, correspondiente al programa de parpadeo simple:

```
# blink_pin5.py
# MicroPython para ESP32 WROOM-32
# Conecta un LED (con resistencia) al pin GPIO5 (GND compartido)

import machine
import time

print("Prueba ESP32 - parpadeo en GPIO5")

led = machine.Pin(5, machine.Pin.OUT)

while True:
    led.value(not led.value()) # toggle
    time.sleep(1)
```

Listing 4: blink_pin5.py – Parpadeo del LED en GPIO5

4.2. Guardar el archivo en el ESP32

Al intentar guardar el programa, Thonny mostrará una ventana preguntando dónde hacerlo. Selecciona la opción **Guardar en el dispositivo Mi-**

croPython.

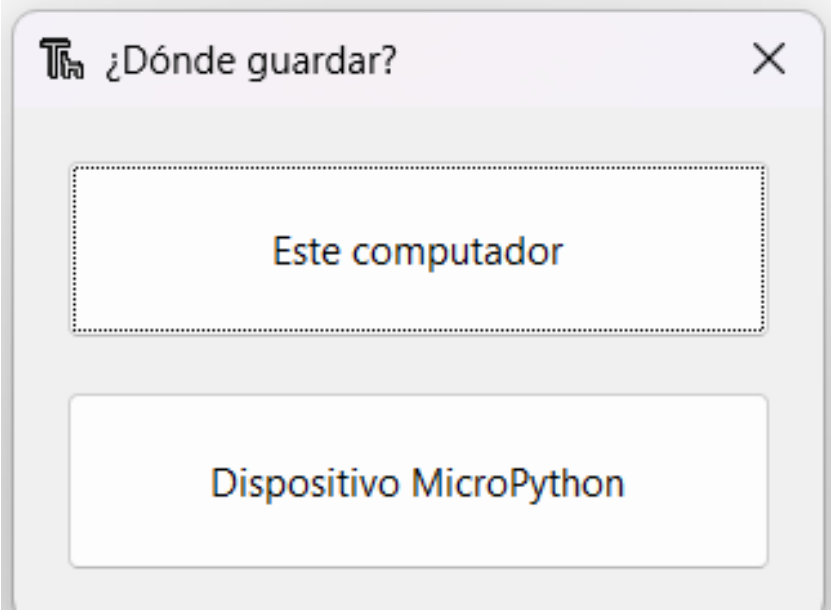


Figura 6: Guardado del archivo directamente en el sistema de archivos del ESP32.

Se recomienda usar nombres cortos y sin espacios, por ejemplo: `blink.py`, `pot.py`, `dht.py`, etc. Los scripts almacenados en la memoria del ESP32 pueden ejecutarse incluso sin conexión al PC (dependiendo de cómo se configuren).

4.3. Ejecución del programa

Para ejecutar el código, presiona el botón **Ejecutar** (ícono de triángulo verde) o usa el atajo `F5`. En la consola inferior se mostrará el texto del programa y el LED conectado al pin GPIO5 comenzará a parpadear con una frecuencia aproximada de 1 Hz.



Figura 7: Ejecución del programa de parpadeo desde Thonny.

4.4. Detención del programa

Para detener la ejecución, presiona el botón **Detener/Interrumpir** (ícono cuadrado rojo) en la barra superior de Thonny. Esto envía una interrupción al bucle principal del programa (equivalente a `Ctrl+C` en la consola REPL).

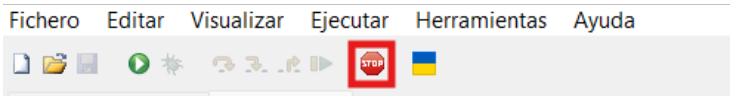


Figura 8: Detención manual del programa en ejecución.

4.5. Ejecución automática al encender el ESP32

MicroPython permite ejecutar automáticamente un script al arrancar el microcontrolador. Para ello, guarda el archivo principal con el nombre `main.py` en la memoria del ESP32. Cada vez que el módulo se energiza o se reinicia, MicroPython busca y ejecuta este archivo.

4.6. Verificación de salida en la consola

Además del parpadeo físico del LED, el programa imprime un mensaje de inicio. En la consola de Thonny se debería observar:

```
Prueba ESP32 - parpadeo en GPIO5
```

Esto confirma que el script fue correctamente transferido, interpretado y ejecutado en la placa.

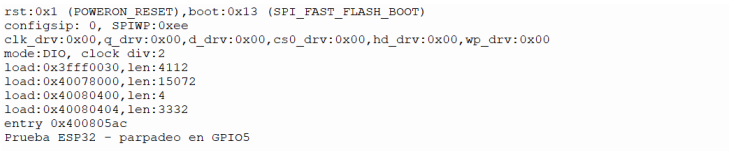


Figura 9: Salida del programa de prueba en la consola de Thonny.

5. Resultados

En esta sección se muestran las evidencias experimentales obtenidas tras la carga y ejecución de los programas desarrollados en el ESP32 mediante Thonny. Cada fotografía corresponde al funcionamiento real de los circuitos descritos en las prácticas.

5.1. Parpadeo del LED en GPIO5

El LED conectado al pin GPIO5 alterna su estado cada segundo, evidenciando el correcto funcionamiento del programa `blink_pin5.py` y la configuración del pin como salida digital.

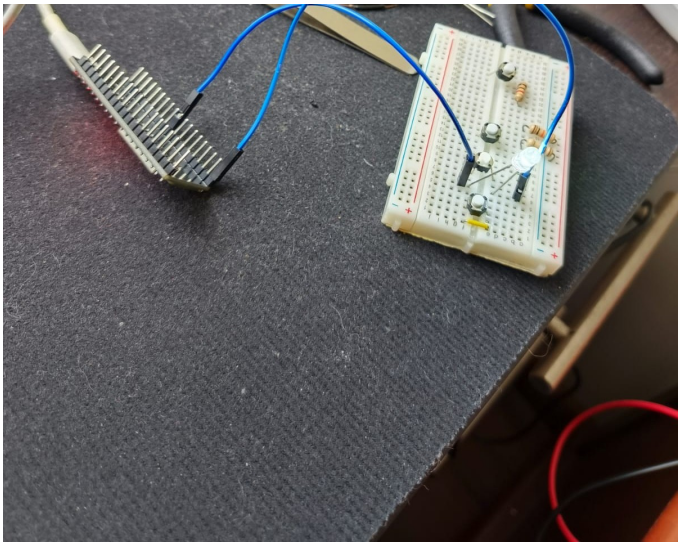


Figura 10: LED conectado al GPIO5 parpadeando con un periodo aproximado de 1 s.

5.2. Control de LED mediante pulsador en GPIO16

En la segunda prueba, el pulsador conectado al pin GPIO16 controla el encendido del LED. El uso del resistor de `pull-up` interno y el manejo de rebote por software permiten una respuesta estable sin parpadeos erráticos.

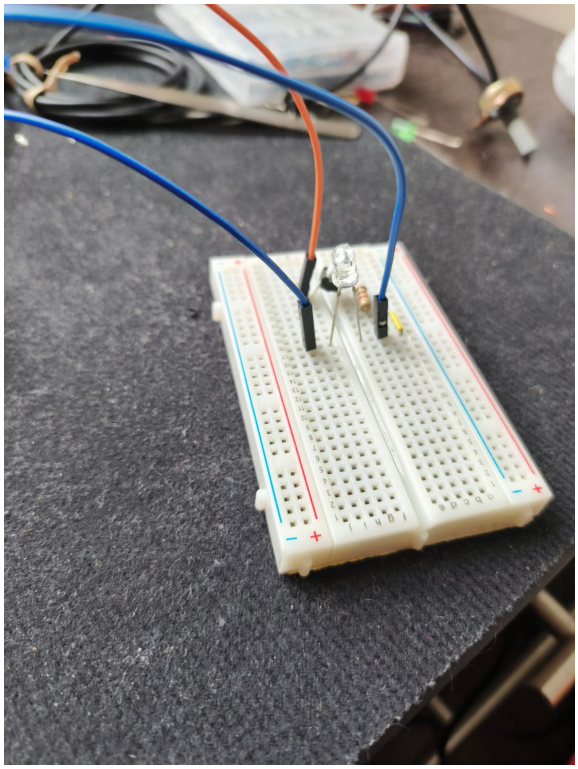


Figura 11: LED controlado por un pulsador (GPIO16), activo al presionar.

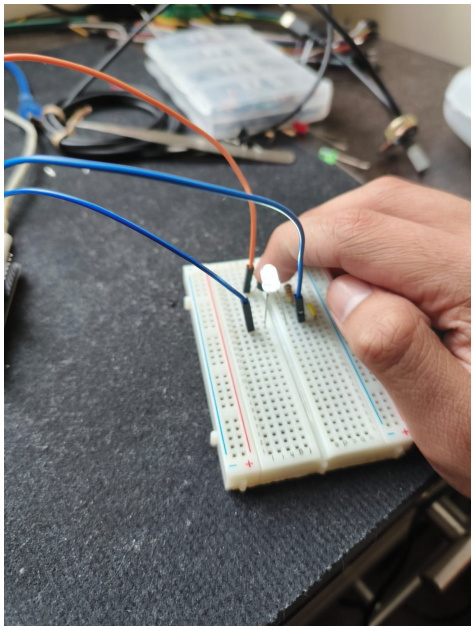


Figura 12: LED controlado por un pulsador (GPIO16), activo al presionar.

5.3. Lectura analógica del potenciómetro en GPIO34

Finalmente, el programa `adc_pot_esp32.py` permite visualizar las lecturas del ADC en la consola de Thonny, mostrando valores proporcionales a la posición del potenciómetro.

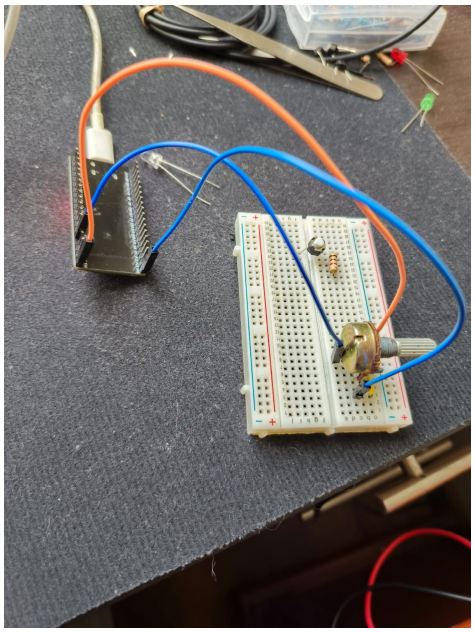


Figura 13: Lectura analógica de un potenciómetro en GPIO34 con salida en la consola.

```
ADC raw:  4095 -> 3.30 V
ADC raw:  4086 -> 3.29 V
ADC raw:  3583 -> 2.89 V
ADC raw:  3312 -> 2.67 V
ADC raw:  2959 -> 2.38 V
ADC raw:  2652 -> 2.14 V
ADC raw:  2370 -> 1.91 V
ADC raw:  1934 -> 1.56 V
ADC raw:  1358 -> 1.09 V
ADC raw:  1168 -> 0.94 V
ADC raw:  1174 -> 0.95 V
ADC raw:  1162 -> 0.94 V
ADC raw:  1189 -> 0.96 V
```

Figura 14: Lectura analógica de un potenciómetro en GPIO34 con salida en la consola.